



第十章 外部排序

计算机学院 杨震

[外部排序的应用对象]

保存在外存储器上的信息量很大的数据记录文件，不可能一次性读入内存。

[外排序与内排序的差别]

- ❖ 内部排序充分利用内存可以随机存取的特点，如
 - 希尔排序中，相隔 d_i 的记录关键字可作比较；
 - 堆排序中，完全二叉树中的父 $R[i]$ 与子 $R[2i]$ 、 $R[2i+1]$ 可比；
 - 快速排序中，需正向和逆向访问记录序列
- ❖ 外存信息的定位和存取受其物理特性的限制

[外部排序的实现手段]

在排序过程中，进行多次内外存之间的数据交换

1 磁带信息的存取

属于顺序存取设备，只适用于存储顺序文件

[工作原理]

磁带不是连续运转设备，而是启停设备

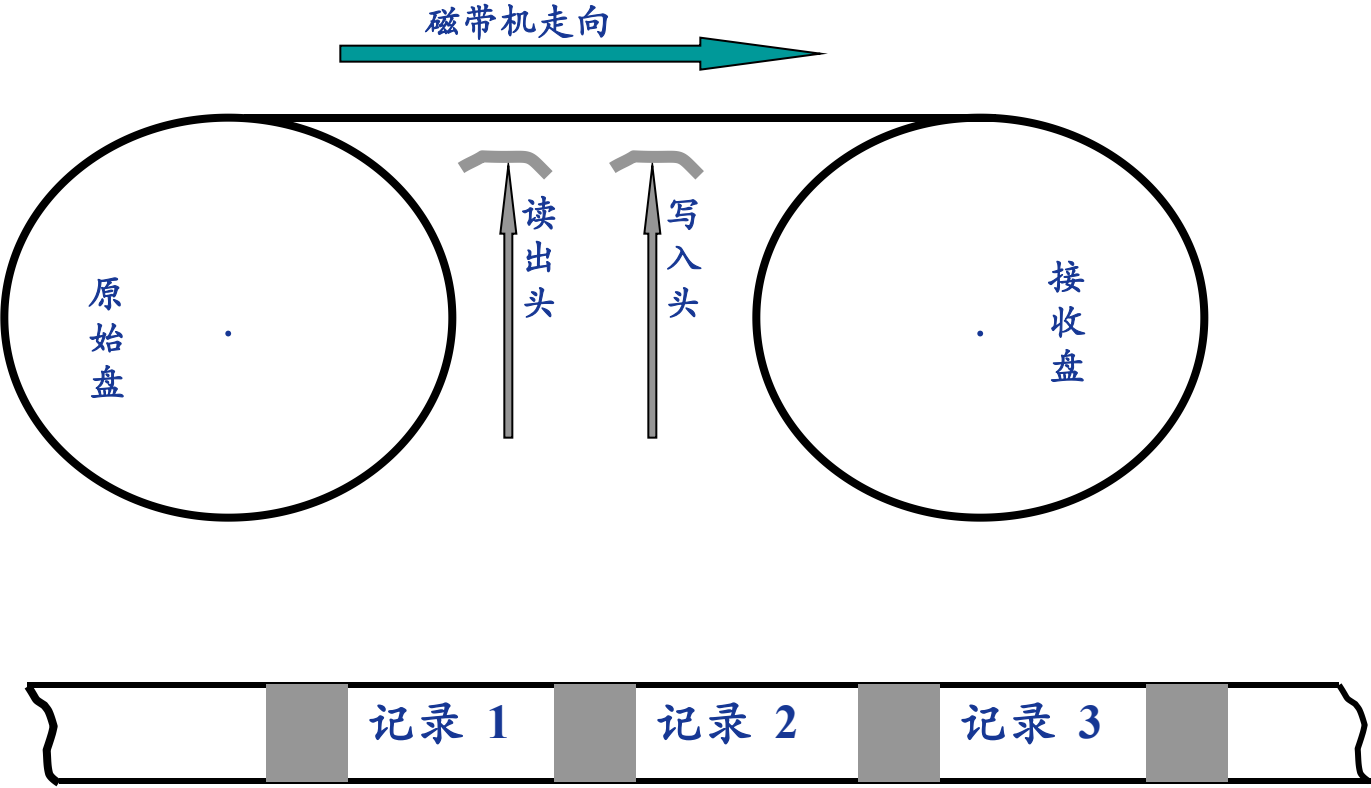
I/O操作以块为单位



读写一块信息的时间 $T_{I/O} = t_a + nt_w$

t_a : 用于磁头定位在物理块起始位置的时间

t_w : 传输一个字符的时间



2 磁盘信息的存取

属于**直接存取设备**，适用于各类存储形式的文件

[工作原理]

磁盘可以是单片的，或有若干个盘片组成盘组

磁道：盘片上的同心圆

柱面：各盘面上半径相同的磁道合在一起

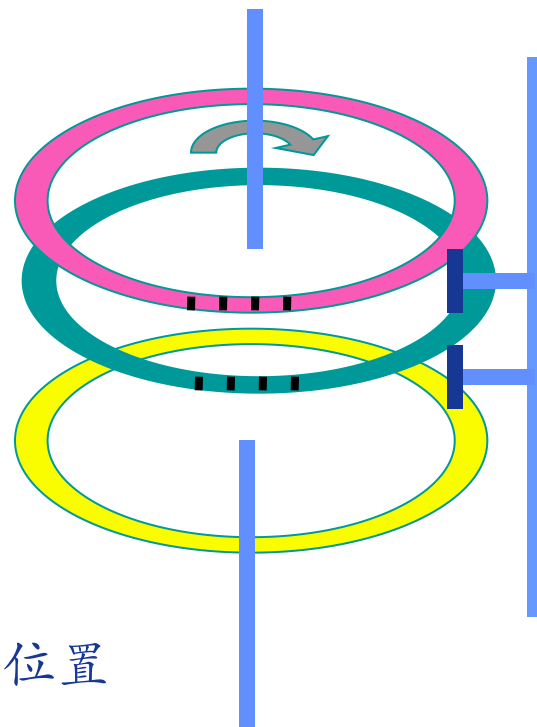
扇区：每个磁道等分为若干扇区(块)

读取信息的过程：

柱面号 移动磁头

盘面号(磁道号) 选定所用磁头

扇区/块号 磁盘一直高速旋转，磁头等待开始位置



读写一块信息的时间 $T_{I/O} = t_{\text{seek}} + t_{\text{la}} + nt_{\text{w}}$

t_{seek} ：磁头定位到磁道的时间

t_{la} ：等待时间

t_{w} ：传输一个字符的时间

外部排序的基本方法—归并排序法

[步骤]

1) 生成若干初始归并串/顺串(文件预处理)

- 把含有 n 个记录的文件，按内存缓冲区大小分成若干长度为 L 的子文件/段；
- 分别调入内存用有效的内排序方法排序后送回外存；

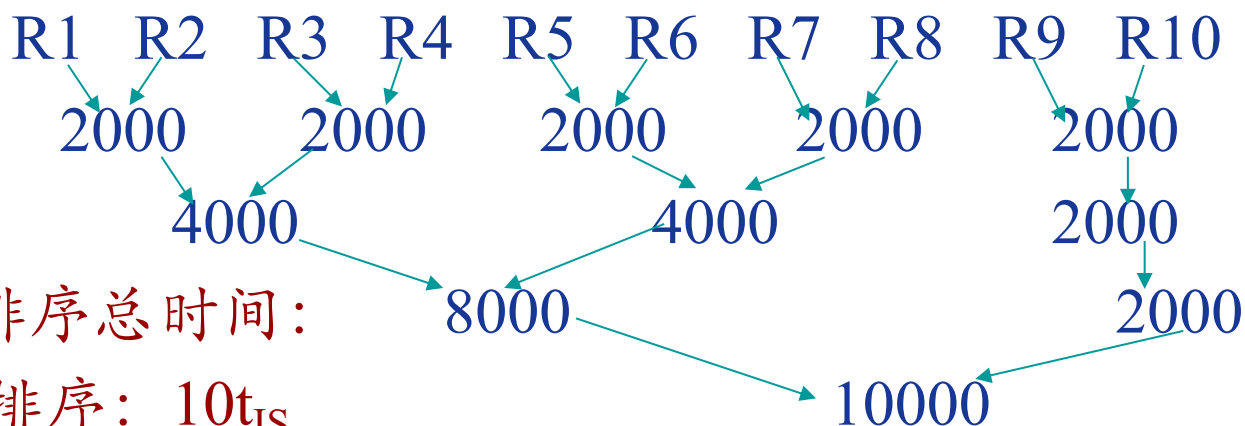
2) 多路合并

对初始归并串逐趟合并，直至最后在外存上得到整个有序文件为止

[例] 某文件共10000个记录，设每个物理块可以容纳200个记录，内存缓冲区可以容纳5个物理块

1) 经过10次内排序后得到10个初始归并段R1~R10

2) 采用两路归并，需四趟可以得到排好序的文件



外排序总时间:

$$\begin{cases}
 \text{内排序: } 10t_{IS} \\
 \text{I/O操作: } 100t_{IO}(\text{排序}) + (100+80+80+100)t_{IO}(\text{归并}) = 460t_{IO} \\
 \text{内部归并: } (10000+8000+8000+10000)t_{mg} = 46000 t_{mg}
 \end{cases}$$

[提高外排序效率的途径]

减少合并趟数 s ，以减少I/O次数

$$s = \lceil \log_k m \rceil$$

- ❖ 扩大初始归并段长度，从而减少初始归并段个数 m
- ❖ 进行多路(k 路)归并

多路(k路)平衡归并

BUPT

[通过简单比较进行归并存在的问题]

设记录总数为 n ，初始归并段的个数为 m ，

从 k 个记录中选择一个关键字最小的记录时的比较次数为 c
则 s 趟内部归并总的比较次数为：

$$C = s \times c(n-1) = \lceil \log_k m \rceil \times c \times (n-1) = \lceil \log_2 m / \log_2 k \rceil \times c \times (n-1)$$

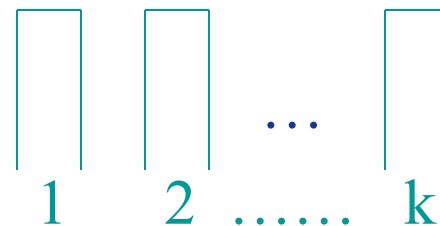
矛盾 $\left\{ \begin{array}{l} k \uparrow, s \downarrow, \text{可减少I/O次数;} \\ k \uparrow, c=(k-1) \uparrow, \text{归并效率} \downarrow \end{array} \right.$

[解决矛盾的方法]

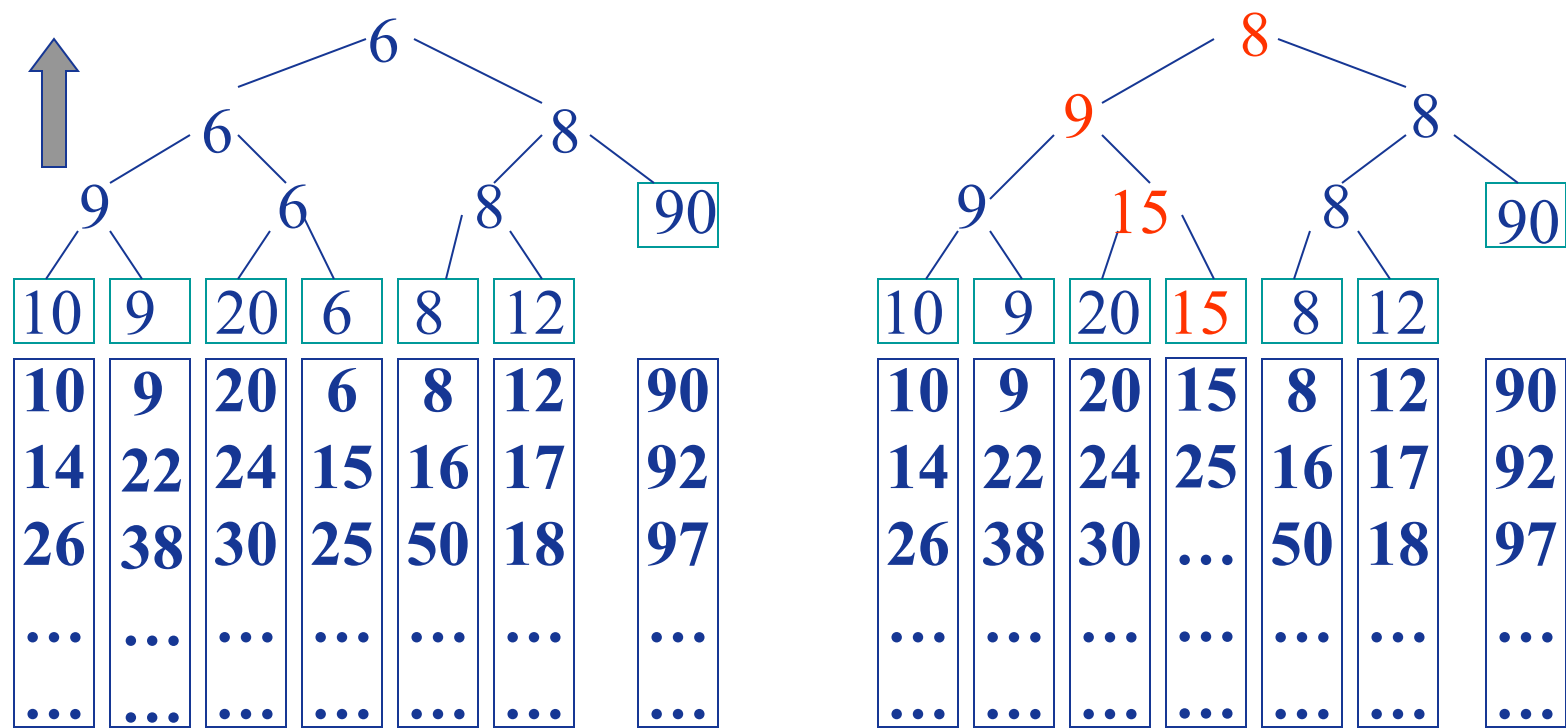
——利用“败者树”选关键字最小的记录

此时 $c = \log_2 k$

则 $C = \lceil \log_2 m \rceil (n-1)$ ，与 k 无关



[胜者树(树形选择排序)的缺陷]

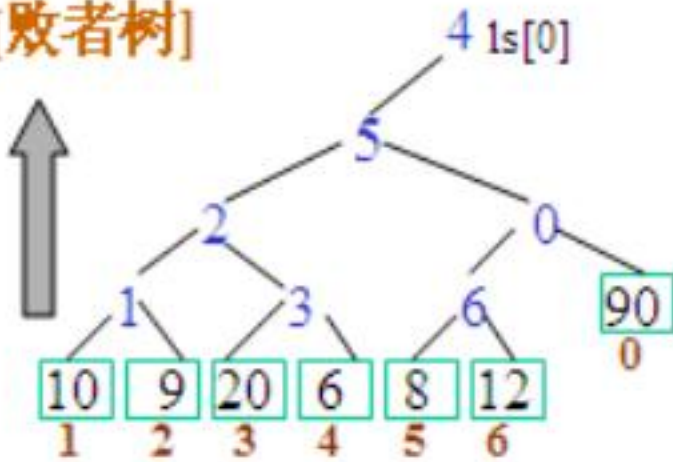


7路合并胜者树

重构后的胜者树

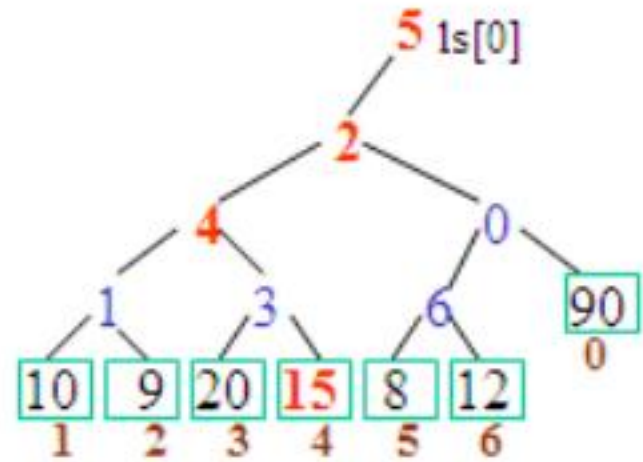
重构胜者树时，从根结点至新补入记录的叶结点的路径上的所有结点都必须更新。

[败者树]



10	9	20	6	8	12	90
14	22	24	15	16	11	92
26	38	30	25	50	18	97
⋮	⋮	⋮	⋮	⋮	⋮	⋮

7路合并败者树



10	9	20	15	8	12	90
14	22	24	25	16	11	92
26	38	30	:	50	18	97
⋮	⋮	⋮		⋮	⋮	⋮

重构后的败者树

败者树的特点：记录败者**编号**，胜者参加下一轮比赛
败者树的优点：重构时修改结点较少。

[数据结构] (依据: 败者树为完全二叉树)

❖ 主: $b[0..k]$

$b[0..k-1]$ —— k 个叶结点 , 存放 k 个输入归并段中当前参加归并的记录

$b[k]$ ——虚拟记录, 该关键字取可能的最小值minkey

❖ 辅: $ls[0..k-1]$ ——不含叶结点的败者树

存放最后胜出的**编号** ($ls[0]$) 以及所记录的**败者编号**

[处理步骤]

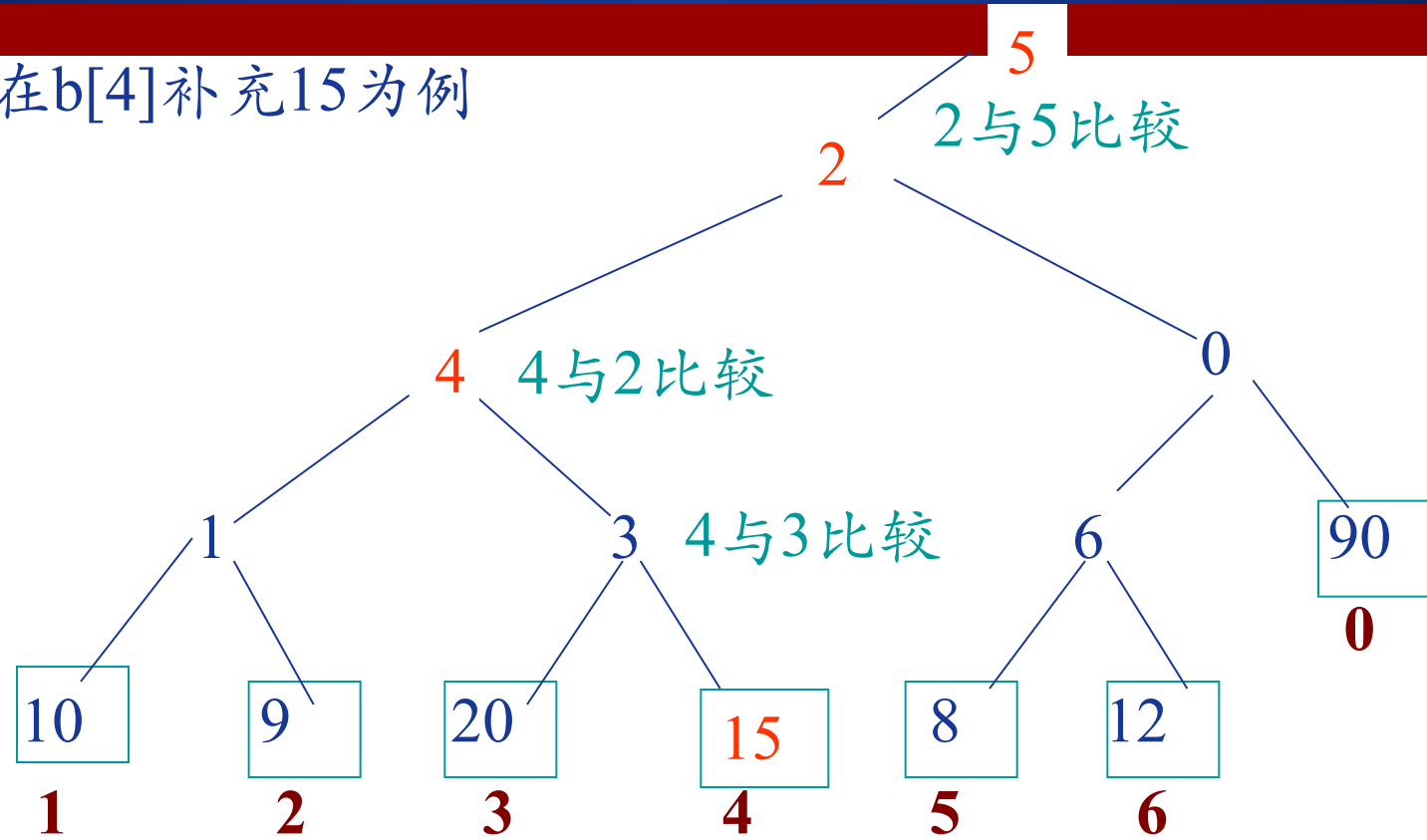
1) 建败者树 $ls[0..k-1]$

2) 重复直至 k 路归并完毕

2.1) 将 $b[ls[0]]$ 写至输出归并段

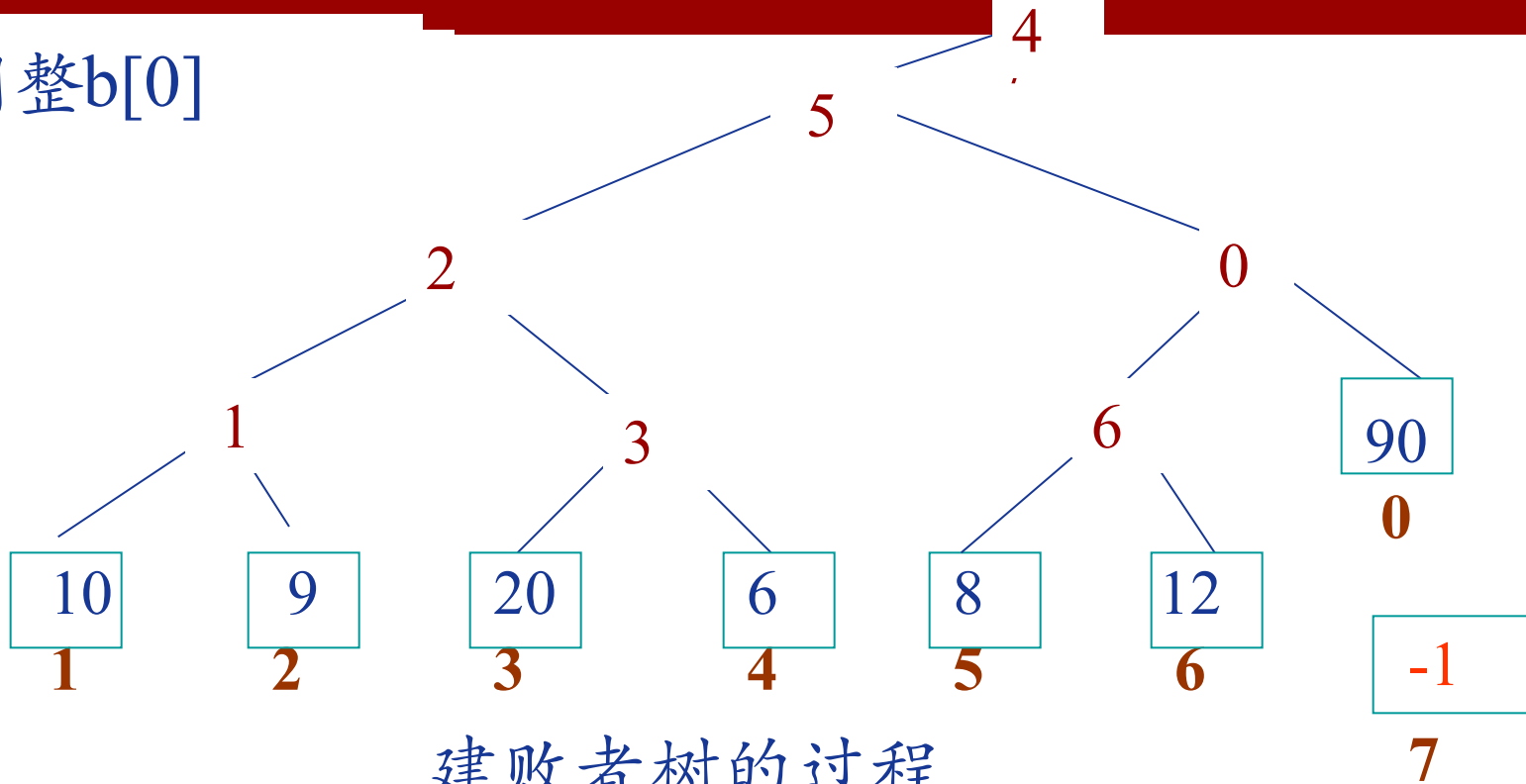
2.2) 补充记录(某归并段变空时, 补 ∞), 调整败者树

以在b[4]补充15为例



调整败者树的方法: 将新补充的结点与其双亲结点比较, 败者留在该双亲结点,胜者继续向上直至树根的双亲

调整b[0]

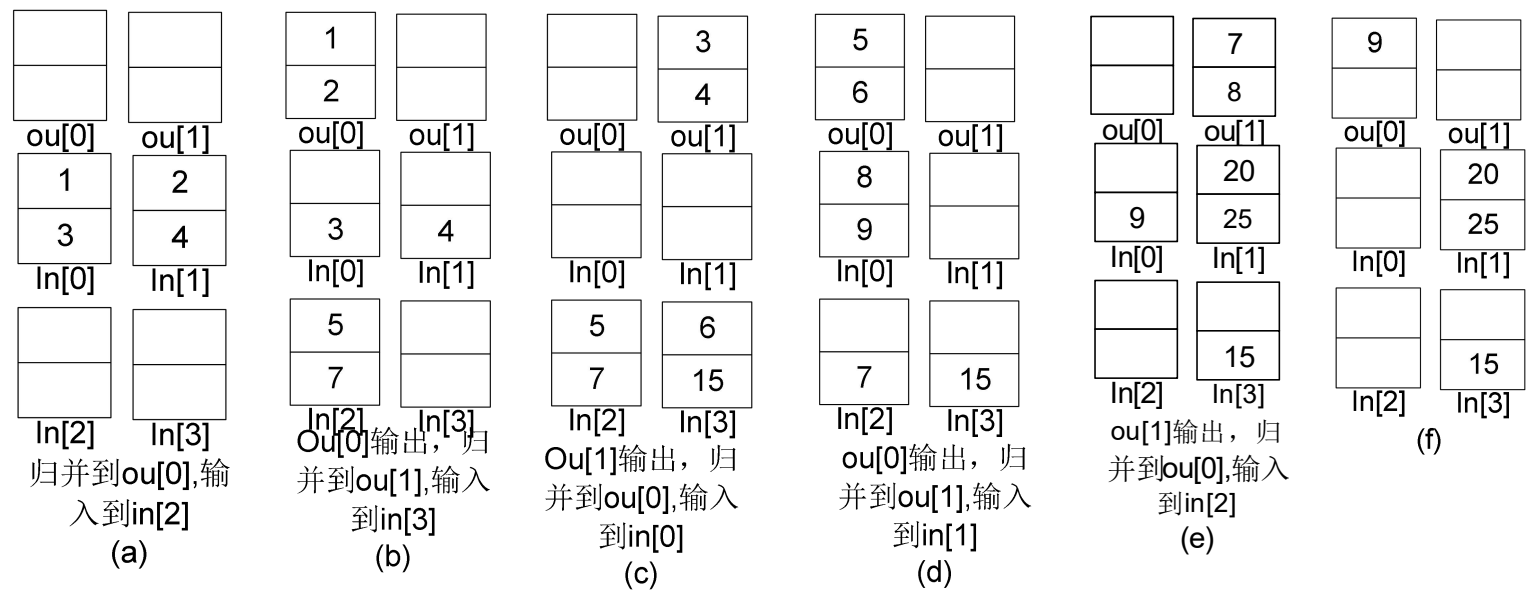


建败者树的过程

[k-路归并对内存的要求]

至少要有k个输入缓冲区和一个输出缓冲区。一般而言，这只是最低要求，并不足以充分提高归并效率。例如：只有一个输出缓冲区，输出缓存正在写磁盘时，内部归并就必须停止工作；若设置两个输出缓冲区，则一个写盘时另一个可用于内部归并存放数据。再如：只有k个输入缓冲区，则当一个输入缓冲区数据处理完成，需从磁盘读数据时，内部归并同样需要等待。因此，最好设2k个输入缓存，但2k个输入缓存也不足以保证能输入输出并行处理。

[示例] 假设两路归并用四个输入缓存in[i], i=0,1,2,3; 两个输出缓存ou[0],ou[1]; 每个缓存能容纳2个记录。假设第0路数据是1,3,5,7,8,9; 第1路数据是2,4,5,15,20,25; 第0路数据用in[0]和in[2], 第1路数据用in[1]和in[3]。(a) 归并开始时, 分别从两路数组的前两个位置将数据读入缓存。(b) in[0]和in[1]归并, ou[0]满, 同时输入第0路数据到in[2]; (c) ou[0]输出, 第1路数据输出到in[3], 归并数据到ou[1], ou[1]满; (d) 继续归并....如 (e) 所示。这时输入ou[1], 第0路输入数据到in[2], 并将归并结果存入到ou[0], 在归并中, 第0路数据用完, 但ou[0]还未放满, 归并过程必须等待第0路的输入结束后才能继续进行。因此, **k路归并时, 如果其中一路数据用完, 必须保证优先为它输入数据。**



败者树的程序实现

```
typedef int  LoserTree[ k ];
typedef struct { KeyType key;
                } Exnode, External[ k+1 ];
```

```
Void K_Merge ( LoserTree &ls,  External & b)
{  for ( i = 0; i < k ; ++i ) input(b[i].key);
  CreateLoserTree(ls);
  { while ( b[ls[0]].key != MAXKEY )
    { q =ls[0]; output(q); input(b[q].key); Adjust( ls, q ); }
    output( ls[0]);
  }
} // K_Merge
```

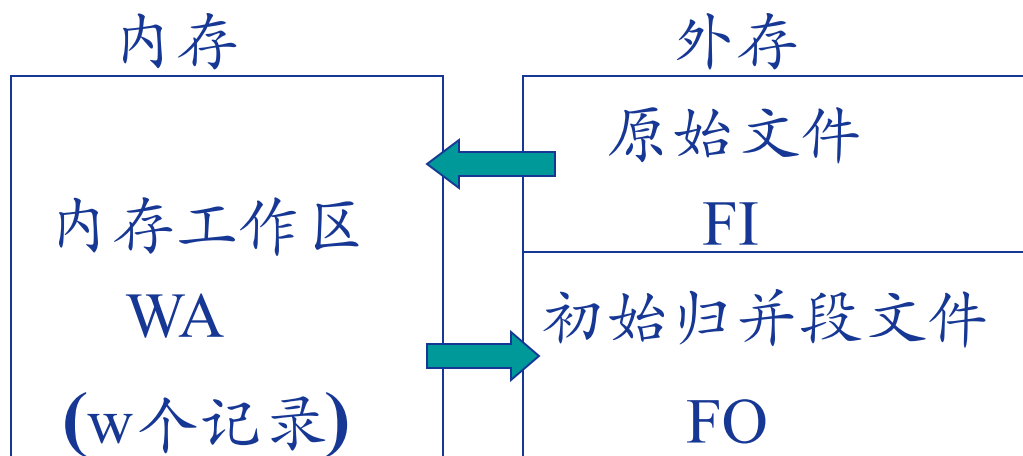

置换-选择排序

BUPT

[目标]

扩大初始归并段长度，突破内存工作区容量（设 w 个记录）的限制。

[置换-选择排序原理]



为简化问题，设每个物理块存放一个记录

- 1) 从FI输入w个记录到工作区WA;
- 2) 在FO中标记一个归并段开始;
- 3) 从WA中选出最小关键字记录, 记为minimax记录;
- 4) 将minimax记录输出到FO中;
- 5) 若FI不空, 则从FI输入下一个记录到WA中;
- 6) 在WA中的关键字比minimax大的记录中选出关键字最小的记录
 - 6.1) 若不能选到, 在FO中标记一个归并段结束;
若由于WA为空而未选出, 则结束处理;
否则, 标记下一个归并段开始, 转 3)
 - 6.2) 否则, 将选出的记录作为新的minimax记录, 转 4)

FO	WA(4个记录)	FI
空	空	36,21,33,87,23,7,62,16, 54,43,29,...
空	36,21,33,87	23,7,62,16,54, 43,29,...
21	36,23,33,87	7,62,16,54, 43,29,...
21,23	36,7,33,87	62,16,54,43,29,...
21,23,33	36,7,62,87	16,54,43,29,...
21,23,33,36	16,7,62,87	54,43,29,...
21,23,33,36,62	16,7,54,87	43,29,...
21,23,33,36,62,87	16,7,54,43	29,...
21,23,33,36,62,87 7	16,29,54,43	...
...		

[置换-选择排序的实现]

利用败者树。

❖ 数据结构

- 主：内存工作区中的记录
 - $b[0..k-1]$ -败者树的外部结点 k 个
- 辅： $ls[0..k-1]$ -不含外部结点的败者树
 - $ls[i]$ -第 i 号结点的淘汰者的编号
 - $ls[0]$ -存放最终的胜者的编号
- $runNum[0..k]$ - $runNum[i]$ 表示 $record[i]$ 所在的段
- $currentNum$ -当前段号
- $winner$ -锦标赛总冠军，取值 $0..k-1$
- $winnerRun$ -记录 $record[winner]$ 的段号
- $maxRun$ -当前即将输出的数据段号
- $minimax$ -最后一条输出记录

调整败者树原则：

段号不相同，段号小者为胜者
段号相同，关键字小的为胜者

❖ 处理步骤

1)初始化败者树：

1.1)所有外部结点置最小值；所有内部结点置最小值；

1.2)对外部结点从编号w-1至0

从外存FI读入记录，置段号为runNum[i]=1，并调整败者树

2)置当前段号currentRun为1；

3)若ls[0]指示的外部结点属于当前段，输出该记录；

否则当前段号=当前段号+1，输出段标后再输出该记录；

4)若FI未空，从FI补充记录，

若补充到b[i]的key<此前输出记录的key(minimax)

则runNum[i]=当前段号+1

否则runNum[i]=当前段号

否则补虚记录：key= ∞ ；runNum[i]=当前段号+1；

5)调整败者树

转3) 直至所有记录被输出

```

void runGeneration( int k ) {
    for(i = 0; i < k; i++){
        readRecord(b[i]); runNum[i]=1;
    }
    initializeLoserTree();//败者树初始化;
    winner = ls[0]; winnerRun =1;
    currentRun =1; maxRun =1;
    while (1) {
        if (winnerRun != currentRun) { //当前段结束
            output(“%d run end”, currentRun);//输出段结束标志;
            if (winnerRun > maxRun) return;
            else currentRun = winnerRun; //更新当前段号
        }
        writeRecord(b[winner]);
        minimax = b[winner].key;
        if (end of input) runNum[winner] = maxRun +1; //没有新记录, 补充虚记录, 虚记录段号=当前最大段号+1
        else { //输入新记录到树
            readRecord(b[winner]);
            if(b[winner].key < minimax)
                runNum[winner] = maxRun = winnerRun +1; //新记录在下一段;
            else runNum[winner] = currentRun;
        }
        winnerRun = runNum[winner];
        for (parent = ( k + winner )/2; parent; parent /=2) //调整败者树
            if(runNum[ls[parent]] < winnerRun || (runNum[ls[parent]] == winnerRun && b[ls[parent]].key < b[winner].key)) //父结点是胜者
                { swap(&winner, &ls[parent]); winnerRun = runNum[winner]; } //交换父子
    } //end while
}
22

```

[置换-选择排序的效果]

- ❖ 所得初始排序段的长度不等
- ❖ 当输入文件记录的关键字大小随机分布时，初始归并段的长度为内存工作区大小 w 的两倍 **思考：为什么？**

[缓冲区及并行操作]

使输入、输出和内部归并三种操作同时进行，也能提高总的外部排序时间。但应综合考虑缓冲区数目、工作区大小等参数的设置。



思考：假设m是所用缓冲区的长度，置换-选择排序法产生文件F（长度为n）的初始归并段的长度至少为多少？最多为多少？

第一个归并段中第一个元素是缓冲区中最小的元素，以后每选一个元素都不应小于前一个选出的元素，故当产生第一个归并段时（即初始归并段），缓冲区中m个元素中除最小元素之外，其他m-1个元素均大于第一个选出的元素，即当以后读入元素均小于输出元素时，初始归并段中也至少能有原有的m个元素。

初始归并段长度最长为n。

平均长度分析：设内存可以存放 m 个记录，则首先从文件读入 m 记录到内存。这 m 个记录肯定可以归入本初始合并段内。这样，再从文件中读入 m 个记录。这 m 个记录中大约平均有一半可以归入本合并段，一半归入下一合并段。因此，合并段的平均长度大约为：

$$m + m/2 + m/4 + \dots = 2m$$

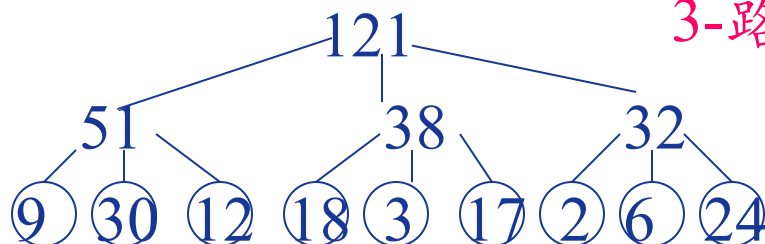
[最佳归并树的引入]

进行k路归并时，初始归并段的不同搭配，会导致归并过程中**I/O的次数**不同。

设有9个初始归并段，记录个数(长度)分别为：

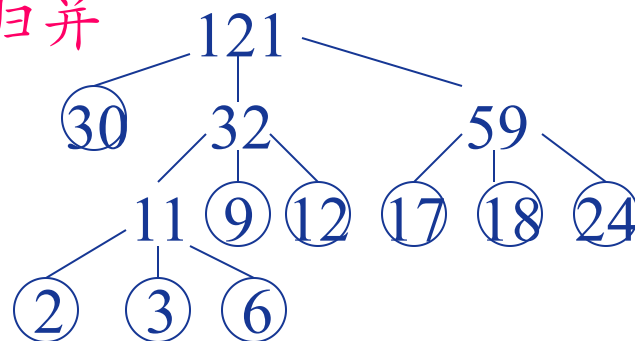
9, 30, 12, 18, 3, 17, 2, 6, 24

方案一



$$(9+30+12+18+3+17+2+6+24) \times 2 + (51+38+32) \times 2 \quad \left\{ \begin{array}{l} I \\ O \end{array} \right. = 484$$

3-路归并



方案二

$$[(2+3+6) \times 3 + (9+12+17+18+24) \times 2 + 30 \times 1] \times 2 = 446$$

所有叶结点加权外通路长度的2倍

[最佳归并树的构造]

最佳归并树即k叉(阶)哈夫曼树。

设初始归并段为m个，进行k-路归并

- 1) 若 $(m-1) \bmod (k-1) \neq 0$ **思考：为什么？如何求得的？**
则需附加 $(k-1) - (m-1) \bmod (k-1)$ 个长度为0的虚段，以使每次归并都可以对应k个段
- 2) 按照哈夫曼树的构造原则（权值越小的结点离根结点越远）构造最佳归并树。

虚段的补法的证明

解：在 k 路平衡归并时，它的归并树的模型是一棵度为 k 的树。在这棵树上的结点要么是叶子，要么是具有 k 个儿子的内部结点。设具有 k 个儿子的内部结点共有 n_k 个

。初始归并段的个数为 m 个。

设 $n = n_k + m$ ，故：

从结点出发的枝条，共计有： $k \times n_k$ 根

若从进入结点结点的角度进行考虑，则共有： $n_k + m - 1$

注意：没有枝条进入根结点。

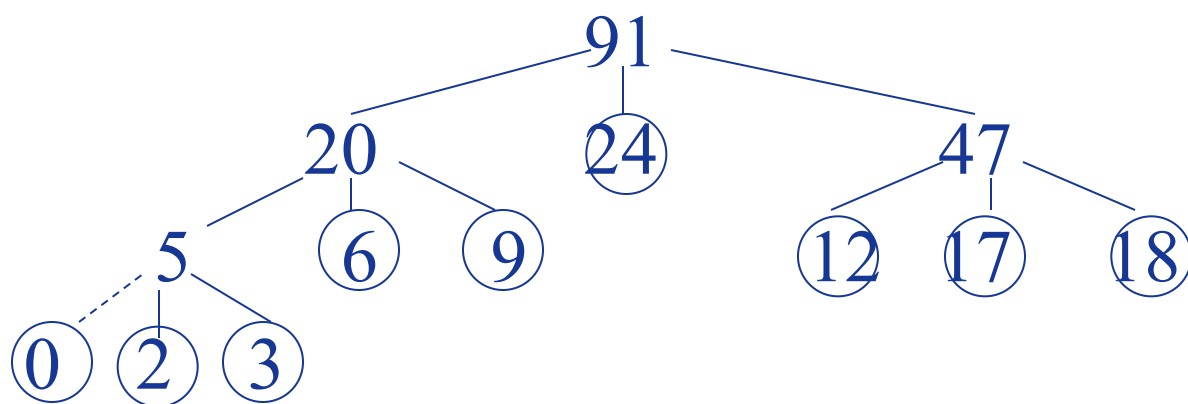
所以， $k \times n_k = n_k + m - 1$

于是： $n_k = (m - 1) / (k - 1)$

这就意味着，若 $(m - 1) \bmod (k - 1) = 0$ ，无需增加虚段。否则，要增加虚段，其

数目为： $(k - 1) - (m - 1) \bmod (k - 1)$ 。

长度分别为2, 3, 6, 9, 12, 17, 18, 24的8个初始归并段进行3路归并, 求最佳归并树。



I/O次数:

$$[(2+3) \times 3 + (6+9+12+17+18) \times 2 + 24 \times 1] \times 2 = 326$$

- ❖ 只有在顺序存取磁带上的数据时，其效能才能充分发挥
- ❖ 本节研究为提高读取信息块的效率，初始归并段在磁带上如何合理分布

... 若干初始归并段

一趟归并的结果 ...

一条带的情况

- ❖ 两种常用的磁带文件排序法
 - 多路平衡归并排序
每一趟归并都是文件中的全部记录参加
 - 多步归并排序
每一步归并不一定需要文件中的全部记录都参加

多路平衡归并排序

[方法]

使用 $2k$ 条磁带进行 k 路归并。开始时，将待归并的 m 个初始归并段均匀的分布在 k 条磁带上。

[示例]

设文件的记录数 $n=4500$ ，
每个物理块250个记录，
内存工作区可容纳750个记录(3块)，
实现 $k=2$ 路归并使用四台磁带机T1、T2、T3、T4，
原数据文件在T4上

排序过程:

- 1) 内排序生成 $4500/750=6$ 个初始归并段, 轮流写在T1和T2上
- 2) T1和T2上的归并段两路合并, 产生的归并段轮流写在T3和T4上
- 3) T3和T4合并, 轮流写在T1和T2上
- 4) 最后合并为一路

T1

T2

T3

T4

$R_1 \dots R_{4500}$

[处理步骤]

从 k 条输入带上平衡地取归并段合并，产生的新的归并段均匀地输出到另 k 条称作输出带的空带上；

下一趟，输入带与输出带的作用互换，……；
直到最后合并成为一个归并段为止。

[$k+1$ 条磁带上的实现方法]

(k 条作输入，1条作输出)

- 1) 初始归并段均匀地写到前 k 条磁带上；
- 2) 进行 k 路归并后的归并段全部写到第 $k+1$ 条磁带上；
- 3) 将第 $k+1$ 条磁带上的归并段再均衡地分配到前 k 条磁带上
- 4) 转2) 进行下一趟合并，直到只有一个归并段为止。

多步归并排序

[方法]

使用 $k+1$ 条磁带进行 k 路归并。巧妙地进行初始分布，使得任何一步开始的时候，总有一条且仅有一条磁带是空着的，可以用它作为本步的输出带。

[示例]

设有17个初始归并段，进行 $k=3$ 路归并。

第 四 步 归 并	T1	E
	T2	
	T3	
	T4	

[如何使初始归并段合理分布]

❖ 方法一：倒推分析

以6条磁带，进行k=5路归并为例：

	T1	T2	T3	T4	T5	T6	总段数
末态→	1	0	0	0	0	0	1
	0	1	1	1	1	1	5
	1	0	2	2	2	2	9
第i步	3	2	0	4	4	4	17
第i-1步	7	6	4	0	8	8	33
	15	14	12	8	0	16	65
	31	30	28	24	16	0	129
	0	61	59	55	47	31	253
	61	0	120	116	108	92	497
初态							

❖ 方法二：利用k-阶广义斐波那契序列

当初始归并段的总数为k-阶广义斐波那契序列中的第j项 $f_j^{(k)}$ 时，在k条磁带上分布的归并段的数目应为：

$$\left\{ \begin{array}{l} t_1^{(k)} = f_{j-1}^{(k)} + f_{j-2}^{(k)} + \dots + f_{j-k}^{(k)} \\ t_2^{(k)} = f_{j-1}^{(k)} + f_{j-2}^{(k)} + \dots + f_{j-k+1}^{(k)} \\ \dots\dots\dots \\ t_{k-1}^{(k)} = f_{j-1}^{(k)} + f_{j-2}^{(k)} \\ t_k^{(k)} = f_{j-1}^{(k)} \end{array} \right.$$

注：若由内部排序得到的初始归并段的总数有不足时，需加“虚段”补充。虚段可以集中在一条带上，也可以分布在多条带上。

$$F_j^{(k)} = 1 \quad (0 \leq j \leq k-1)$$

$$F_j^{(k)} = F_{j-k}^{(k)} + F_{j-k+1}^{(k)} + \dots + F_{j-1}^{(k)} \quad (j \geq k)$$

1. 假设一次I/O的物理块大小为150，每次可对750个记录进行内部排序，那么对含有150000个记录的磁盘文件进行4-路平衡归并排序时，共需进行多少次I/O？
2. 已知某文件经过置换-选择排序之后，得到长度分别为47，9，31，18，4，12，23和7（单位均为物理块）的8个初始归并段。试为3-路平衡归并设计一个读写外存次数最少的归并方案，并求出读写外存的次数。